Ivett Anahi Zaragocin Mora

Temporizadores del ESP32

AUTORRECARGA · API · INTERRUPCIONES

Autorrecarga y desbordamiento



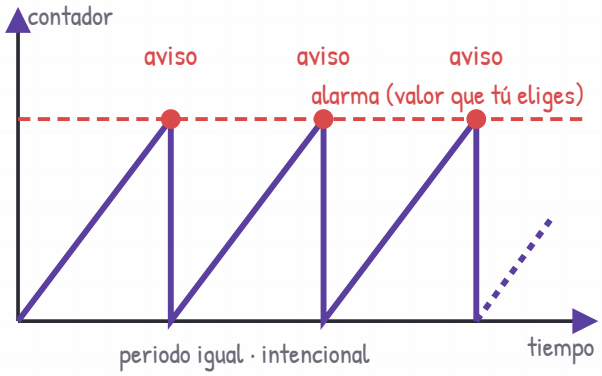
El comparador revisa en cada tick; al llegar a la alarma avisa y el contador vuelve a 0

Autorrecarga: al alcanzar la alarma, el contador **vuelve solo a 0**; el aviso se repite a intervalos iguales.

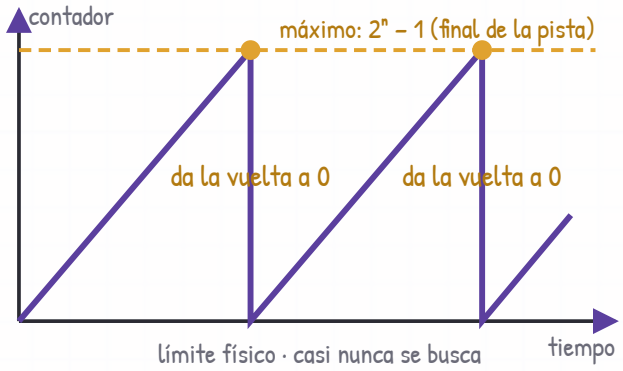
tiempo entre avisos = alarma × duración del tick

Ej.: 500 000 ticks × 1 µs = 0,5 s

ALARMA CON AUTORRECARGA



DESBORDAMIENTO



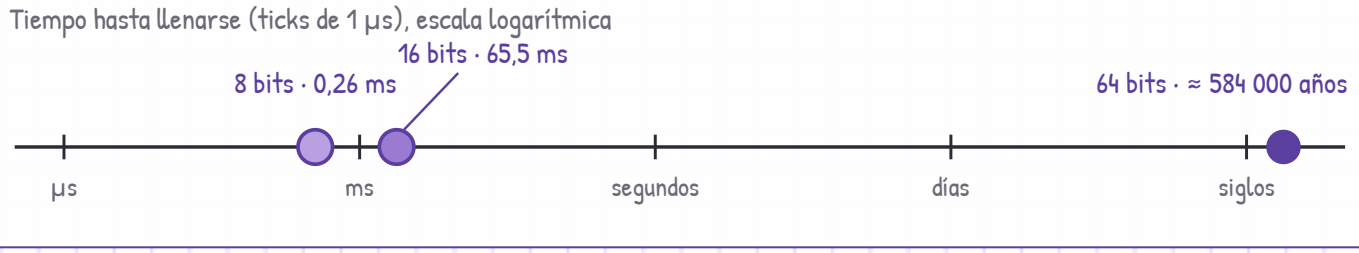
Mismo contador, distinto tope: la alarma la fijas tú; el desbordamiento lo impone el hardware

	Alarma con autorrecarga	Desbordamiento
Valor que lo provoca	Uno que tú eliges	El máximo de los bits ($2^n - 1$)
Qué pasa	Aviso y vuelta a 0	El contador «da la vuelta» a 0
¿Intencional?	Sí, uso normal	No, límite físico

Límite del contador

$$\text{con } n \text{ bits, el máximo es } 2^n - 1$$

Contador	Valor máximo	Se llena (ticks de 1 µs)
8 bits (Timer0 AVR)	255	0,26 ms
16 bits (Timer1 AVR)	65 535	65,5 ms
64 bits (ESP32)	18 446 744 073 709 551 615	≈ 584 000 años



⚠️ En el ESP32 el desbordamiento **prácticamente no ocurre**: se trabaja con la alarma.

Temporizadores del ESP32

4 TEMPORIZADORES **2 GRUPOS**

GRUPO TIMG0

Timer 0	Timer 1
contador de 64 bits prescaler de 16 bits	contador de 64 bits prescaler de 16 bits

GRUPO TIMG1

Timer 0	Timer 1
contador de 64 bits prescaler de 16 bits	contador de 64 bits prescaler de 16 bits

- Cada uno cuenta **hacia arriba o hacia abajo** y tiene **autorrecarga**.
- Trabajan en paralelo: hasta **4 avisos periódicos independientes**.
- `TIMG0` y `TIMG1` = «Timer Group» 0 y 1.

ESP32 frente al ATmega328P

Característica	ESP32 (WROOM-32)	ATmega328P (Uno)
Temporizadores	4	3 (Timer0, 1, 2)
Contador	64 bits	8 o 16 bits
Reloj de entrada	80 MHz	16 MHz
Prescaler	Cualquier N de 2 a 65 535	Solo 1, 8, 64, 256 o 1024
Se llena (sin prescaler)	Prácticamente nunca	Timer1: 65 536 ÷ 16 MHz ≈ 4,1 ms

$$\text{tick} = 80\ \text{MHz} \div \text{prescaler}$$

Prescaler 80 → 1 MHz → **tick de 1 µs**.

API

API (*Application Programming Interface*): funciones ya programadas que se usan **sin saber cómo están hechas por dentro**. Ej.: `pinMode()`, `digitalWrite()`, `millis()`.



» No escribiremos los registros del temporizador a mano, pero hay que saber **que existen y qué guardan**.

Core y versión 3.x

Core paquete que se instala en el Arduino IDE para programar **una placa concreta**; trae el compilador y la API de esa placa.

DE FÁBRICA

Solo core de placas Arduino (Uno, Mega). **No programa el ESP32.**

PARA EL ESP32

Core de Espressif: `esp32 by Espressif Systems`.

Herramientas → **Placa** → **Gestor de placas** → **esp32 by Espressif Systems**

1. **Qué cambió** Con la **versión 3.x** (3.0, 3.1…) cambiaron nombres y parámetros de las funciones de los temporizadores.
2. **Consecuencia** Un código de la versión 2 **no compila** en la 3.

⚠️ La mayoría de tutoriales usan la versión 2: copiarlos da **errores de compilación**.

Interrupción, ISR y callback

Interrupción señal de un periférico que dice a la CPU *«detente, pasó algo»*; ejecuta una función y **vuelve adonde estaba**.

ISR (*Interrupt Service Routine*): la función que atiende la interrupción.

	Del temporizador (semana 3)	Externa (semana 4)
La dispara	El temporizador llega a la alarma	Un pin cambia de nivel
Cuándo	Cada cierto tiempo, predecible	Cuando pasa algo, sin aviso
Ejemplo	LED cada 2 segundos	Pulsar un botón

Línea de tiempo de la CPU



Callback función que tú escribes pero **no llamas tú**: se la entregas a otro para que la llame cuando ocurra algo.

1 Escribes la función
lo que debe pasar al sonar la alarma.

2 Se la entregas
al temporizador, con una función de la API.

3 El hardware la llama
cada vez que el contador llega a la alarma.

Mientras corre la ISR, `loop()` espera: la ISR debe ser muy corta; lo lento va después (patrón bandera).

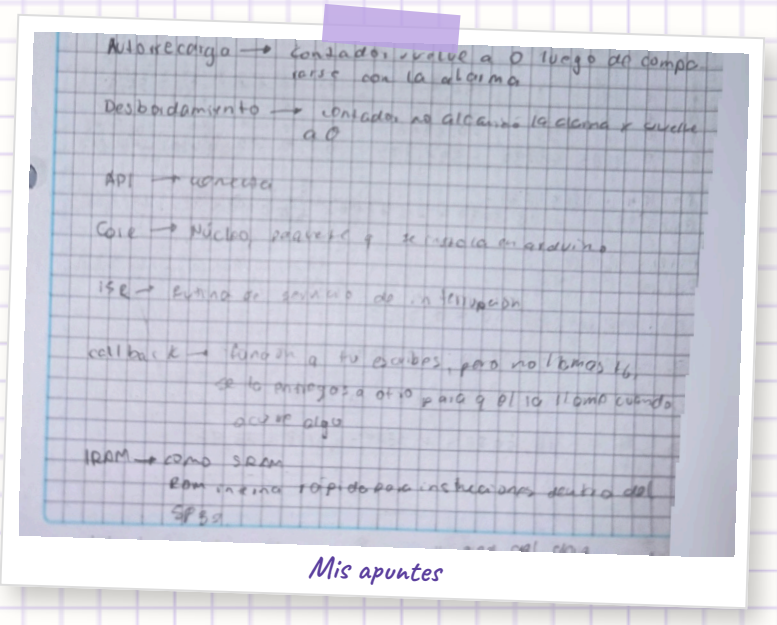
IRAM_ATTR y volatile

MEMORIA FLASH
Guarda el programa. Grande y persistente, pero **lenta y a veces ocupada**.

IRAM
RAM interna rápida para instrucciones. **Siempre disponible**.

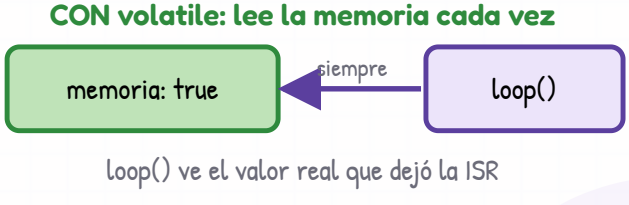
IRAM_ATTR etiqueta delante del nombre de la ISR: pide al compilador **copiarla a la IRAM**.

```
void IRAM_ATTR onTimer() {
  // ... código muy corto ...
}
```


Mis apuntes

volatile el compilador **optimiza** y no ve que la ISR cambia la variable; `volatile` obliga a leerla de memoria cada vez.






```
volatile bool alarma = false;    // compartida entre la ISR y loop()
```

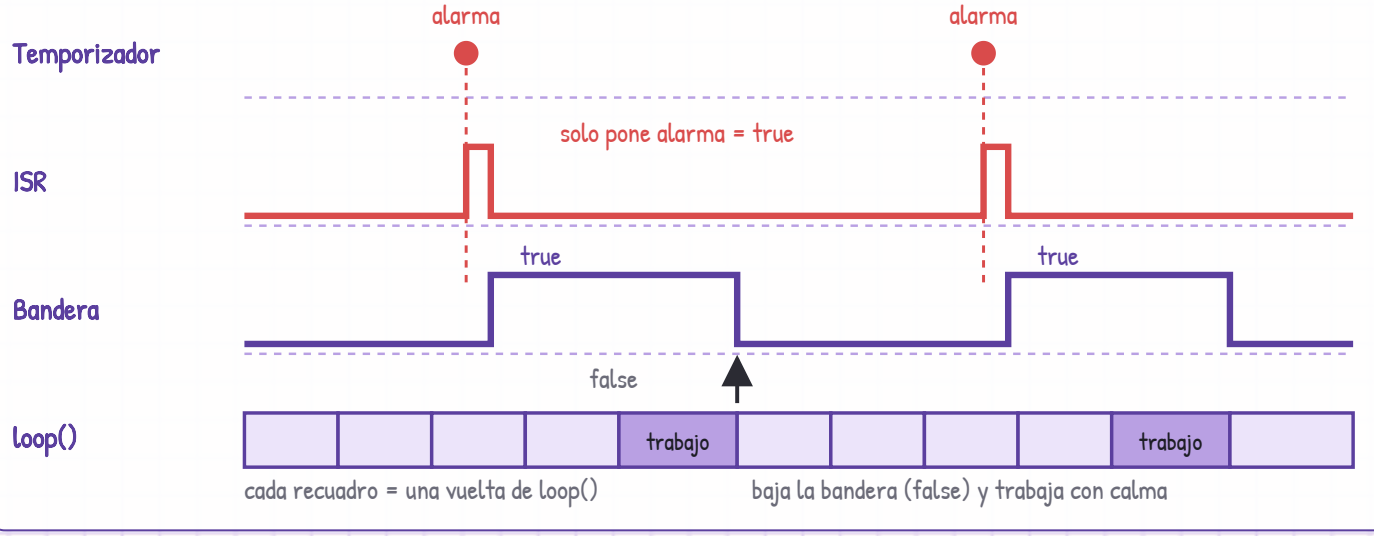
Patrón bandera

1. La ISR solo avisa

Pone la **bandera** en `true` y termina.

2. loop() trabaja

Si la bandera está arriba, la baja (`false`) y hace el trabajo lento.



✦ Ejemplo completo (core 3.x)

```
hw_timer_t *temporizador = NULL;
volatile bool alarma = false;  // bandera

void IRAM_ATTR onTimer() {      // ISR
  alarma = true;                // solo avisa
}

void setup() {
  Serial.begin(115200);
  temporizador = timerBegin(1000000);
  timerAttachInterrupt(temporizador,
                       &onTimer);
  timerAlarm(temporizador,
             500000, true, 0);
}

void loop() {
  if (alarma) {             // ¿bandera?
    alarma = false;         // bájala
    Serial.println("0,5 s");
  }
}
```

✦ Lectura del código

1. `volatile` : bandera compartida ISR-`loop()` .
2. `IRAM_ATTR` : la ISR vive en la IRAM.
3. La ISR (callback) solo hace `alarma = true` .
4. La API fija el tick (1 µs), el callback y la alarma de 500 000 ticks con autorrecarga.
5. `loop()` baja la bandera y trabaja.

RESUMEN RÁPIDO

- **Autorrecarga**: al llegar a la alarma el contador vuelve a 0; **desbordamiento**: límite físico $2^n - 1$.
- **ESP32**: 4 timers (2 grupos × 2), 64 bits, 80 MHz, prescaler de 16 bits.
- Core 3.x cambió la API de timers: el código de la versión 2 no compila.
- ISR con `IRAM_ATTR` , variables compartidas `volatile` ; la ISR solo levanta la bandera y `loop()` trabaja.